Week3

생성일	@2026년 3월 29일 오후 2:00
예습과제	

05 GBM

GBM의 개요 및 원리

- **부스팅 알고리즘**: 여러 개의 **약한 학습기(weak learner)**를 순차적으로 학습-예측하며, 잘못 예측한 데이터에 **가중치 부여**를 통해 오류를 개선해 나가는 방식.
- **에이다부스트(AdaBoost)**: 오류 데이터에 가중치를 부여하면서 부스팅을 수행하는 대표적 알고리즘.
  - **Step 1~5**: 첫 번째 학습기가 분류 후 오류 데이터 발생 -> 오류 데이터에 가중치 부여 -> 다음 학습기가 더 잘 분류할 수 있게 크기 조정 -> 모든 예측 결정 기준을 결합해 최종 예측 수행.
  - **결과**: 개별 약한 학습기보다 결합된 모델의 정확도가 훨씬 높음.
- **GBM(Gradient Boost Machine)**: 에이다부스트와 유사하나, 가중치 업데이트에 **경사 하강법(Gradient Descent)**을 이용하는 것이 차이점.
  - **오류식**: h(x) = y - F(x)
  - **방향성**: 반복 수행을 통해 오류식을 최소화하는 방향으로 가중치 업데이트 값을 도출.

GBM의 특징 및 수행 시간

- **수행 성능**: 일반적으로 랜덤 포레스트보다 예측 성능이 뛰어난 경우가 많음.
- **단점**: 수행 시간이 오래 걸리고 하이퍼 파라미터 튜닝 노력이 더 필요함.
  - **병렬 처리 미지원**: 약한 학습기가 순차적으로 학습하므로 멀티 CPU 코어 시스템에서도 병렬 처리가 지원되지 않음.

GBM 하이퍼 파라미터

- **loss**: 경사 하강법에서 사용할 비용 함수 지정.

- **learning_rate**: 가중치 보정에 적용하는 계수(0~1 사이). 너무 작으면 수행 시간이 길어지고, 너무 크면 최소 오류 값을 찾지 못할 수 있음.
- **n_estimators**: weak learner의 개수. 많을수록 성능이 좋아질 수 있으나 수행 시간이 오래 걸림.
- **subsample**: 학습에 사용하는 데이터 샘플링 비율. 과적합 방지를 위해 1보다 작은 값으로 설정 가능.

---

06 XGBoost

XGBoost의 개요 및 장점

- **특징**: GBM 기반이나 느린 수행 시간, 과적합 규제 부재 등의 문제를 해결.
- **주요 장점**:
  - **뛰어난 예측 성능:** 분류와 회귀 영역에서 뛰어난 성능 발휘
  - **빠른 수행 시간:** 병렬 수행 및 다양한 기능으로 GBM 대비 빠른 속도 보장
  - **과적합 규제:** 자체적인 과적합 규제 기능 보유
  - **Tree pruning:** 긍정 이득이 없는 분할을 가지치기하여 분할 수 감소
  - **자체 내장 교차 검증:** 반복마다 교차 검증 및 조기 중단 기능 수행
  - **결손값 자체 처리:** 결손값을 자체적으로 처리 가능

주요 하이퍼 파라미터

- **부스터 파라미터**:
  - **eta (learning_rate)**: 학습률. 파이썬 래퍼 디폴트 0.3, 사이킷런 래퍼 디폴트 0.1.
  - **min_child_weight**: 과적합 조절을 위한 데이터 weight 총합. 클수록 분할 자제.
  - **gamma**: 리프 노드 분할을 결정할 최소 손실 감소 값.
  - **max_depth**: 트리 최대 깊이. 보통 3~10 사이 적용.
  - **lambda (reg_lambda)** / **alpha (reg_alpha)**: L2 / L1 Regularization 적용 값.
- **학습 태스크 파라미터**:
  - **objective**: 최솟값을 가져야 할 손실 함수 정의 (binary:logistic, multi:softmax 등).
  - **eval_metric**: 검증에 사용되는 함수 (rmse, logloss, error 등).

조기 중단(Early Stopping)

- **개념**: `n_estimators`에 도달하지 않더라도 예측 오류가 개선되지 않으면 학습을 중지하여 수행 시간 개선.
- **설정**: `early_stopping_rounds` 파라미터를 통해 최소 반복 횟수 지정.

---

07 LightGBM

LightGBM의 특징

- **장점**: XGBoost보다 학습 시간이 훨씬 적고 메모리 사용량도 적음.
- **단점**: 10,000건 이하의 적은 데이터 세트 적용 시 **과적합** 발생 가능성 높음.
- **트리 분할 방식**: **리프 중심 트리 분할(Leaf Wise)** 방식 사용.
  - 기존의 균형 트리 분할(Level Wise)과 달리, 최대 손실 값을 가지는 리프 노드를 지속적으로 분할.
  - 비대칭적이고 깊은 트리가 생성되나, 반복 학습 시 예측 오류 손실 최소화 가능.

주요 하이퍼 파라미터

- **num_leaves**: 하나의 트리가 가질 수 있는 최대 리프 개수. 모델 복잡도 제어의 주요 파라미터.
- **min_data_in_leaf (min_child_samples)**: 리프 노드가 되기 위한 최소 레코드 수. 과적합 제어용.
- **max_depth**: 트리 깊이 제한. 리프 중심 방식이므로 상대적으로 깊게 설정됨.

---

08 베이지안 최적화 기반 HyperOpt

베이지안 최적화 개요

- **정의**: 목적 함수 식을 모르는 블랙 박스 형태 함수에서 최적 입력값을 적은 시도로 효과적으로 찾는 방식.
- **구성 요소**:
  - **대체 모델(Surrogate Model)**: 관측된 데이터를 기반으로 최적 함수 추정 (가우시안 프로세스 또는 TPE 사용).

- **획득 함수(Acquisition Function)**: 대체 모델 결과를 바탕으로 다음으로 관측할 하이퍼 파라미터 계산.

HyperOpt 사용법 및 유의사항

- **로직 구성**: 입력 변수 및 검색 공간 설정 -> 목적 함수 설정 → `fmin()` 함수로 최솟값 유추.
- **유의사항**:
  - HyperOpt는 목적 함수 반환값의 **최솟값**을 찾음. 정확도처럼 클수록 좋은 지표는 -1을 곱해 반환해야 함.
  - 검색 공간 입력값은 실수형이므로, 정수형 하이퍼 파라미터(max_depth 등) 적용 시 **형변환(int)** 필수.

---

10 분류 실습 - 캐글 신용카드 사기 검출

데이터 특징 및 불균형 분포

- **레이블 불균형**: Class 속성이 0(정상)과 1(사기)로 분류되며, 전체 데이터 중 사기 트랜잭션은 약 **0.172%**에 불과함.
- **문제점**: 이상 데이터 건수가 너무 적어 다양한 유형을 학습하지 못하고 정상 레이블로 치우친 학습을 수행하게 됨.

언더 샘플링과 오버 샘플링

- **언더 샘플링**
  - 많은 레이블을 가진 데이터 세트를 적은 레이블 수준으로 감소시킴.
  - 정상 레이블 데이터를 너무 많이 감소시켜 학습이 제대로 안 될 수 있음.
- **오버 샘플링**
  - 적은 레이블 데이터를 증식하여 충분한 데이터를 확보.
  - 단순히 복제하면 과적합이 발생하므로 피처 값을 약간 변경하여 증식.
- **SMOTE**
  - K 최근접 이웃(KNN)을 찾아 기존 데이터와 약간의 차이가 나는 새로운 데이터를 생성하는 방식.

데이터 가공 및 변환

- **의미 없는 피처 제거**: 데이터 생성 작업용 속성인 'Time' 피처 등은 제거.
- **로그 변환**: 데이터 분포도가 심하게 왜곡된 경우 넘파이의 **log1p()** 함수를 이용해 왜곡을 상당 수준 개선.
- **LGBMClassifier 설정**: 레이블 불균형이 심할 경우 `boost_from_average=False`로 설정해야 재현율 및 ROC-AUC 성능 저하를 막을 수 있음.

이상치 데이터 제거 (IQR 방식)

- **IQR(Inter Quantile Range)**: 사분위(Quantile) 값의 편차를 이용하는 기법.
- **계산 및 판단 기준**
  - **IQR** = Q3(75% 지점) - Q1(25% 지점)
  - **최댓값 기준**: Q3 + (IQR * 1.5)
  - **최솟값 기준**: Q1 - (IQR * 1.5)
  - 이 범위를 초과하거나 미달하는 데이터를 이상치로 간주.
- **피처 선택**: 결정값과 가장 상관성이 높은 피처를 위주로 이상치를 검출하는 것이 효율적.

11 스태킹 앙상블

스태킹(Stacking)의 개념

- **정의**: 개별 알고리즘으로 예측한 데이터를 기반으로 다시 예측을 수행하는 방식.
- **특징**
  - 개별 모델의 예측 결과 세트를 최종 메타 데이터 세트로 만들어 별도의 ML 알고리즘으로 최종 학습을 수행.
  - 캐글과 같은 대회에서 아주 미세한 성능 수치를 높여야 할 때 자주 사용됨.
  - 일반적으로 성능이 비슷한 모델들을 결합할 때 나은 성능 향상을 도출할 수 있음.

스태킹 모델의 구성

- **개별 기반 모델**: 1차적으로 학습과 예측을 수행하는 여러 개의 모델.
- **최종 메타 모델**: 기반 모델들이 반환한 예측 데이터를 학습 피처로 사용하여 최종 예측을 수행.

CV 세트 기반의 스태킹

- **목적**: **과적합** 문제 개선.
- **프로세스**
  - **Step 1**: 개별 모델별로 원교차 검증을 통해 학습용 & 테스트용 스태킹 데이터를 생성.
  - **Step 2**: 생성된 데이터를 모두 합쳐 최종 학습용 데이터 세트와 테스트용 데이터 세트를 생성.